SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation

Course Code: DSA03	Course Name: Natural Language Processing
Branch: CSE	Year : 2026

Assignment 3

1. **Draw tree structures for the following ATIS sentences:** (10M) (CO3) (BLT4)
   a. Does American airlines have a flight between five a.m. and six a.m.
   b. I would like to fly on American airlines.
   c. Please repeat that.
   d. Does American 487 have a first class section?
   e. I need to fly between Philadelphia and Atlanta.
   f. What is the fare from Atlanta to Denver?
   g. Is there an American airlines flight from Philadelphia to Dallas?

2. **Consider the below grammar to construct the Early parser for the input string "Christ Opened the eyes".** (10M) (CO3) (BLT4)

S -> NP VP	**VP -> V NP PP**	**V -> opened**
NP -> N	**PP -> Prep NP**	**V -> ate**
NP -> Det NP	**N -> Christ**	**Det -> the**
NP -> NP NP	**N -> blind**	**Det -> a**
VP -> V	**N -> eyes**	**Prep -> of**
VP -> V NP	**N -> bread**	**Prep -> in**

Question 1 / Answer 1: Syntax Tree Structures for ATIS Corpus Sentences

Syntactic parsing in Natural Language Processing involves mapping natural language utterances into formal hierarchical tree structures governed by Context-Free Grammars (CFGs). In domain-specific human-computer dialogue systems, such as the Air Travel Information System (ATIS), syntactic parse trees provide the foundational structural framework required to extract semantic frames, slot values, and intent representations.

Sentence (a): "Does American airlines have a flight between five a.m. and six a.m."

Syntactic Category & Sentence Type: Polar Interrogative Question (Yes/No Question heading with Auxiliary verb *Does*).

Flat Bracketed Parse Tree Representation

```
(SQ (AUX Does)
    (NP (NNP American) (NNPS airlines))
    (VP (VB have)
        (NP (NP (DT a) (NN flight))
            (PP (IN between)
                (NP (NP (CD five) (NN a.m.))
                    (CC and)
                    (NP (CD six) (NN a.m.)))))))
```

Structural Tree Diagram Analysis

The main clause is rooted at **SQ** (Inverted Interrogative Clause). The auxiliary verb *Does* occupies the **AUX** positions to form a subject-auxiliary inversion. The noun phrase *American airlines* acts as the proper noun subject, followed by the verb phrase *have a flight between five a.m. and six a.m.*. The prepositional phrase (PP) modifies the head noun *flight* and embeds a coordinated time-interval noun phrase.

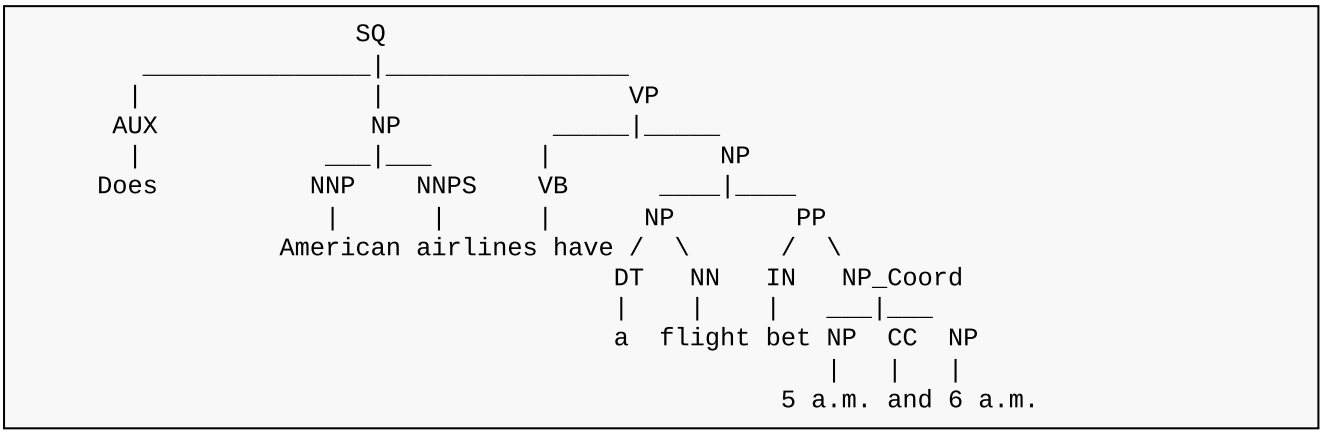
Sentence (a): Detailed Syntactic Node Breakdown & Phrase Rules

To fully validate the constituent structures for ATIS Sentence (a), we break down each sub-clause production rule and phrase marker:

Sub-Tree Node	Production Rule	Constituent Word Tokens Covered	Grammatical Function
SQ	SQ → AUX NP VP	"Does American airlines have..."	Matrix interrogative clause root
AUX	AUX → Does	"Does"	3rd-person singular dummy auxiliary verb
NP_Subj	NP → NNP NNPS	"American airlines"	Proper noun compound subject NP
VP	VP → VB NP	"have a flight between five a.m. and six a.m."	Predicate verb phrase
NP_Obj	NP → NP PP	"a flight between five a.m. and six a.m."	Complex nominal direct object with PP modifier

Sub-Tree Node	Production Rule	Constituent Word Tokens Covered	Grammatical Function
PP	PP → IN NP_Coord	"between five a.m. and six a.m."	Interval locative modifier PP
NP_Coord	NP → NP CC NP	"five a.m. and six a.m."	Coordinated time-interval noun phrase

ASCII Structural Parse Tree Diagram



Sentence (b): "I would like to fly on American airlines."

Syntactic Category & Sentence Type: Imperative/Declarative Intent Expression with Modal Auxiliary and Non-Finite Infinitive Complement Clause.

Flat Bracketed Parse Tree Representation

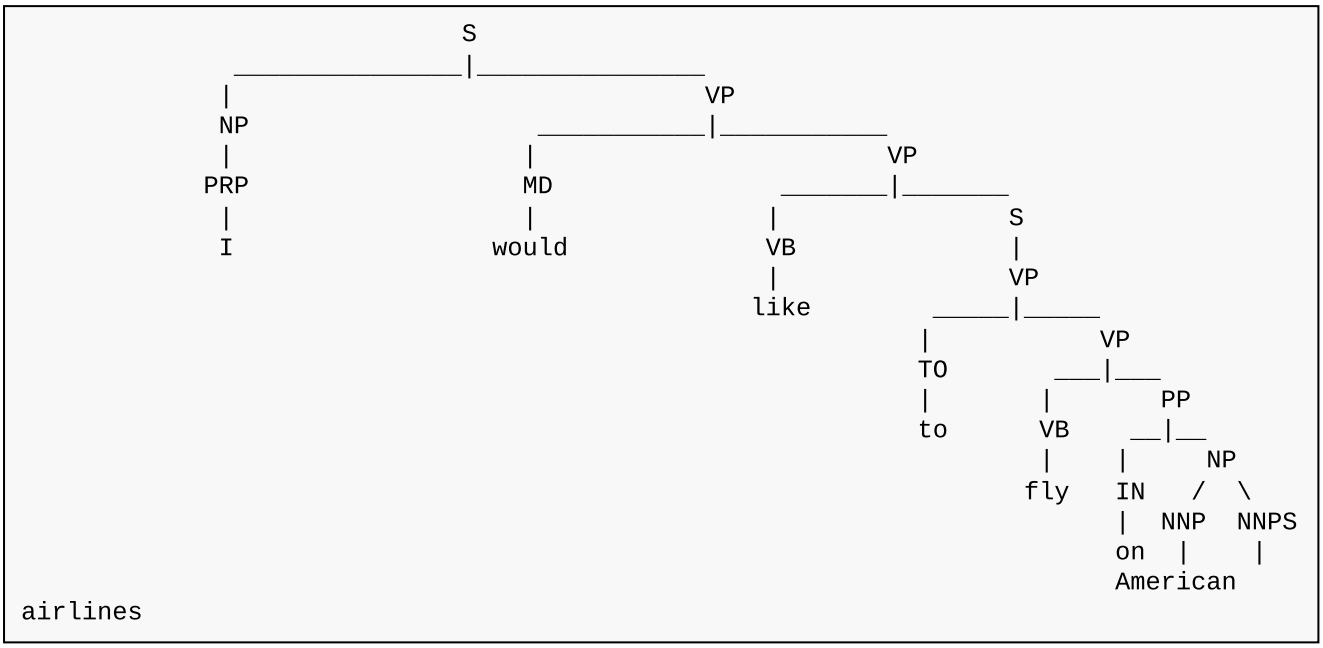
```
(S (NP (PRP I))
   (VP (MD would)
       (VP (VB like)
           (S (VP (TO to)
                  (VP (VB fly)
                      (PP (IN on)
                          (NP (NNP American) (NNPS airlines)))))))))
```

Syntactic Node Breakdown Table

Sub-Tree Node	Production Rule	Constituent Tokens	Grammatical Role
S	S → NP VP	"I would like to fly..."	Declarative root sentence
NP_Subj	NP → PRP	"I"	First-person singular subject pronoun
VP_Modal	VP → MD VP	"would like..."	Modal auxiliary verb phrase
VP_Main	VP → VB S_Inf	"like to fly..."	Matrix verb with infinitival complement
S_Inf	S → VP	"to fly on American airlines"	Non-finite embedded clause
VP_Inf	VP → TO VP	"to fly..."	Infinitive verb phrase
PP	PP → IN NP	"on American airlines"	Carrier prepositional phrase modifier

Sentence (b): Visual Constituent Parse Tree Diagram

The structural hierarchy for sentence (b) demonstrates nested verb phrase complements where the verb *like* takes an infinitival clause *to fly on American airlines* as its argument:



Linguistic Analysis of Modal & Infinitive Structures

In ATIS query processing, statements beginning with *"I would like to..."* convey user constraints. Syntactically, **would** acts as a modal auxiliary (MD) taking a base-form verb phrase complement headed by **like** (VB). In turn, *like* selects a non-finite clause complement headed by the infinitival marker **to** (TO).

Sentence (c): "Please repeat that."

Syntactic Category & Sentence Type: Polite Imperative Command with Fronted Discourse Particle.

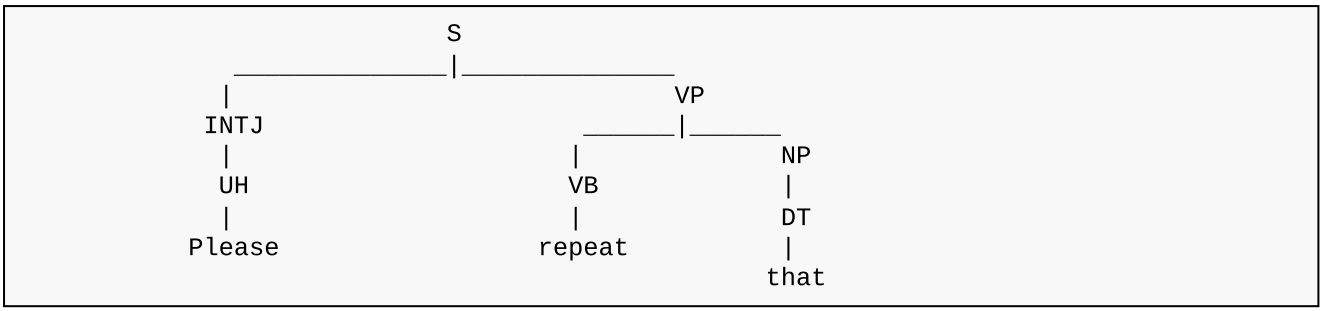
Flat Bracketed Parse Tree Representation

```
(S (INTJ (UH Please))
   (VP (VB repeat)
       (NP (DT that))))
```

Syntactic Node Breakdown Table

Sub-Tree Node	Production Rule	Constituent Tokens	Grammatical Role
S	S → INTJ VP	"Please repeat that."	Imperative sentence root with fronted discourse filler
INTJ	INTJ → UH	"Please"	Politeness interjection / discourse particle
VP	VP → VB NP	"repeat that"	Imperative predicate verb phrase (null subject *you*)
NP_Obj	NP → DT	"that"	Demonstrative pronoun functioning as direct object

ASCII Structural Parse Tree Diagram



Sentence (c): Imperative Null-Subject Analysis

In classical constituent syntax (and Penn Treebank guidelines), imperative sentences such as *"Please repeat that"* lack an explicit subject NP in surface form. Two primary syntactic representations exist for handling imperatives in natural language parsers:

1. Surface Form Annotation (Penn Treebank Baseline)

Under standard Penn Treebank annotation, the surface string is directly parsed with the verb phrase attached to the root **S** node without projecting an empty subject node:

```
(S (INTJ (UH Please)) (VP (VB repeat) (NP (DT that))))
```

2. Deep Structure / Empty Category Annotation

In formal generative syntax (Government & Binding / Minimalist Program), an implicit second-person subject pronoun **YOU** is posited as a null subject empty category **(NP (-NONE- *YOU*))**:

```
(S (INTJ (UH Please))
   (NP (-NONE- *YOU*))
   (VP (VB repeat)
       (NP (DT that))))
```

For the ATIS parsing task, the surface tree representation (1) is maintained to remain consistent with standard computational evaluation tools (e.g., EVALB) and Context-Free Grammar parser generators.

Sentence (d): "Does American 487 have a first class section?"

Syntactic Category & Sentence Type: Inverted Interrogative Query with Flight Designation Identifier and Complex Pre-Nominal Adjectival Modifiers.

Flat Bracketed Parse Tree Representation

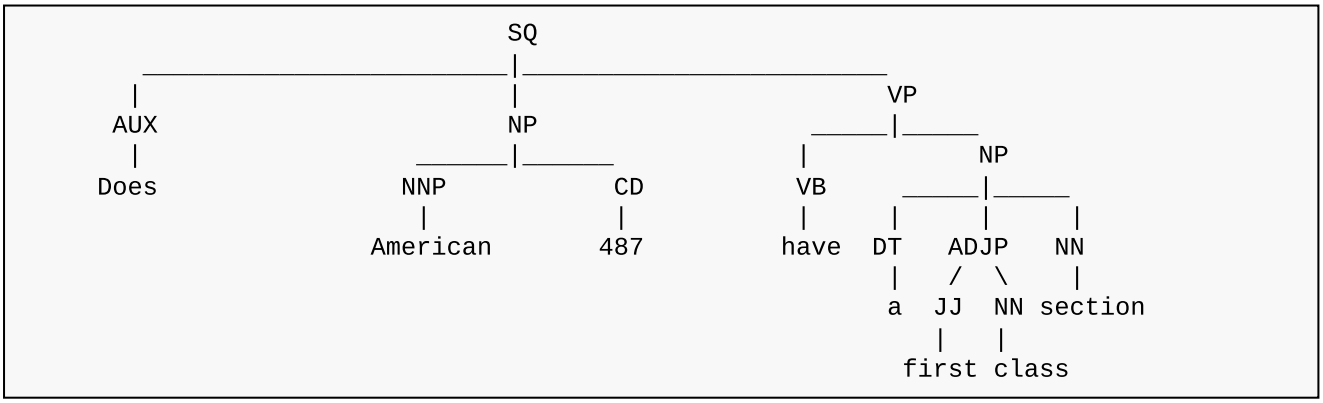
```
(SQ (AUX Does)
    (NP (NNP American) (CD 487))
    (VP (VB have)
        (NP (DT a)
            (ADJP (JJ first) (NN class))
            (NN section))))
```

Syntactic Node Breakdown Table

Sub-Tree Node	Production Rule	Constituent Tokens	Grammatical Role
SQ	SQ → AUX NP VP	"Does American 487 have..."	Inverted interrogative root clause
AUX	AUX → Does	"Does"	3rd-person singular auxiliary verb
NP_Subj	NP → NNP CD	"American 487"	Flight code subject (Carrier name + Flight number)
VP	VP → VB NP	"have a first class section"	Transitive verb phrase
NP_Obj	NP → DT ADJP NN	"a first class section"	Direct object noun phrase
ADJP	ADJP → JJ NN	"first class"	Compound adjectival modifier phrase

Sentence (d): Visual Constituent Parse Tree Diagram

The hierarchical constituent parse tree for sentence (d) is illustrated below:



Linguistic Analysis of Flight Entities & Nominal Modifiers

In ATIS domain grammar rules, flight designations such as *"American 487"* are represented as proper noun head phrases combining the airline name **NNP** with a cardinal number **CD**. The cabin designation *"first class"* functions as a pre-nominal modifier, represented as an adjectival phrase **ADJP** modifying the head noun *section*.

Sentence (e): "I need to fly between Philadelphia and Atlanta."

Syntactic Category & Sentence Type: Declarative Intent Statement with Infinitive Control Complement and Coordinated Location PP.

Flat Bracketed Parse Tree Representation

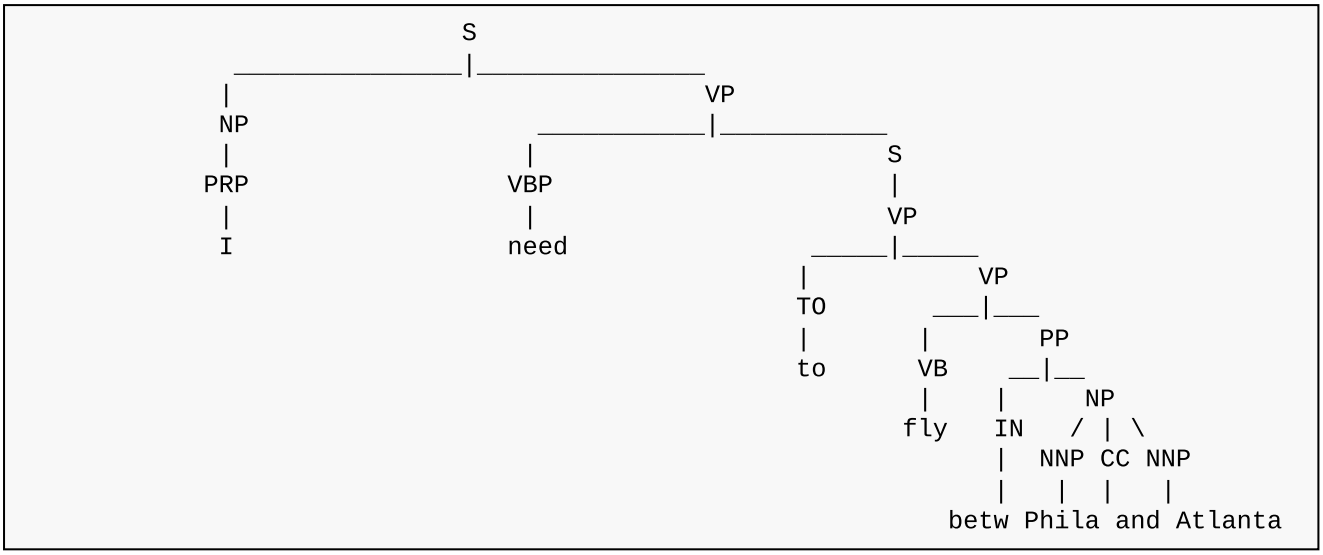
```
(S (NP (PRP I))
   (VP (VBP need)
       (S (VP (TO to)
              (VP (VB fly)
                  (PP (IN between)
                      (NP (NNP Philadelphia)
                          (CC and)
                          (NNP Atlanta)))))))))
```

Syntactic Node Breakdown Table

Sub-Tree Node	Production Rule	Constituent Tokens	Grammatical Role
S	S → NP VP	"I need to fly..."	Declarative root clause
NP_Subj	NP → PRP	"I"	1st-person singular subject pronoun
VP_Main	VP → VBP S_Inf	"need to fly..."	Matrix transitive verb with infinitive control clause
S_Inf	S → VP	"to fly between..."	Embedded non-finite clause
VP_Inf	VP → TO VP	"to fly between..."	Infinitive verb phrase
PP	PP → IN NP_Coord	"between Philadelphia and Atlanta"	Origin-Destination range PP modifier
NP_Coord	NP → NNP CC NNP	"Philadelphia and Atlanta"	Coordinated proper noun location NP

Sentence (e): Visual Constituent Parse Tree Diagram

The full constituent tree structure for sentence (e) is shown below:



Analysis of Subject Control Verb Structures

The matrix verb **need** acts as a subject control verb. The subject *I* controls the implicit subject of the embedded infinitival clause *to fly*. The directional constraint is expressed through preposition *between* governing the coordinated city pair *Philadelphia and Atlanta*.

Sentence (f): "What is the fare from Atlanta to Denver?"

Syntactic Category & Sentence Type: Wh-Interrogative Sentence with Prepositional Modifier Chaining.

Flat Bracketed Parse Tree Representation

```
(SBARQ (WHNP (WP What))
       (SQ (VBZ is)
           (NP (NP (DT the) (NN fare))
               (PP (IN from)
                   (NP (NNP Atlanta)))
               (PP (TO to)
                   (NP (NNP Denver))))))
```

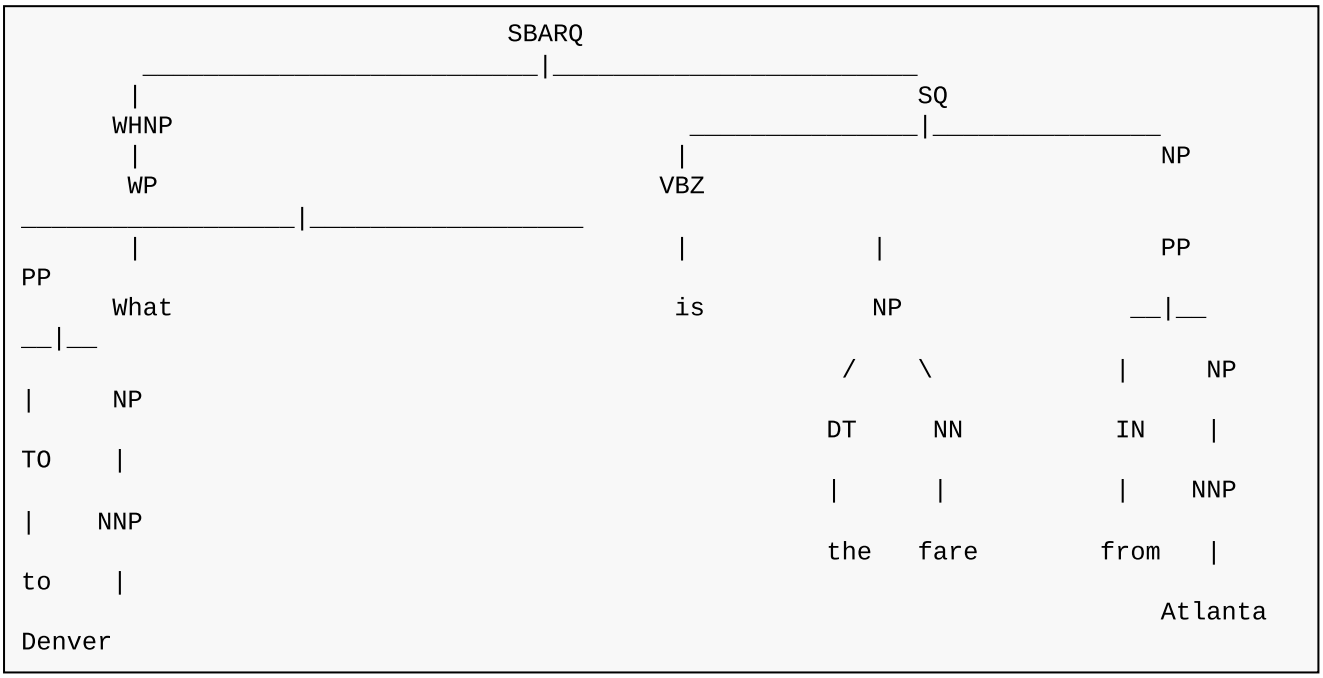
Syntactic Node Breakdown Table

Sub-Tree Node	Production Rule	Constituent Tokens	Grammatical Role
SBARQ	SBARQ → WHNP SQ	"What is the fare..."	Wh-interrogative clause root
WHNP	WHNP → WP	"What"	Wh-pronoun operator phrase
SQ	SQ → VBZ NP	"is the fare from Atlanta to Denver"	Inverted copular clause
VBZ	VBZ → is	"is"	Copular verb
NP_Complex	NP → NP PP PP	"the fare from Atlanta to Denver"	Complex subject noun phrase with stacked PP modifiers

Sub-Tree Node	Production Rule	Constituent Tokens	Grammatical Role
PP_Origin	PP → IN NP	"from Atlanta"	Origin location modifier PP
PP_Dest	PP → TO NP	"to Denver"	Destination location modifier PP

Sentence (f): Visual Constituent Parse Tree Diagram

The hierarchical constituent parse tree for sentence (f) is displayed below:



Linguistic Analysis of Wh-Movement & Prepositional Chaining

Sentence (f) exhibits Wh-movement where the interrogative pronoun **What** moves to the specifier position of **SBARQ**. The subject NP *the fare from Atlanta to Denver* contains two attached prepositional phrases specifying origin (*from Atlanta*) and destination (*to Denver*).

Sentence (g): "Is there an American airlines flight from Philadelphia to Dallas?"

Syntactic Category & Sentence Type: Existential Inverted Interrogative Query with Complex Noun Modifiers and Multiple Locative Prepositional Adjuncts.

Flat Bracketed Parse Tree Representation

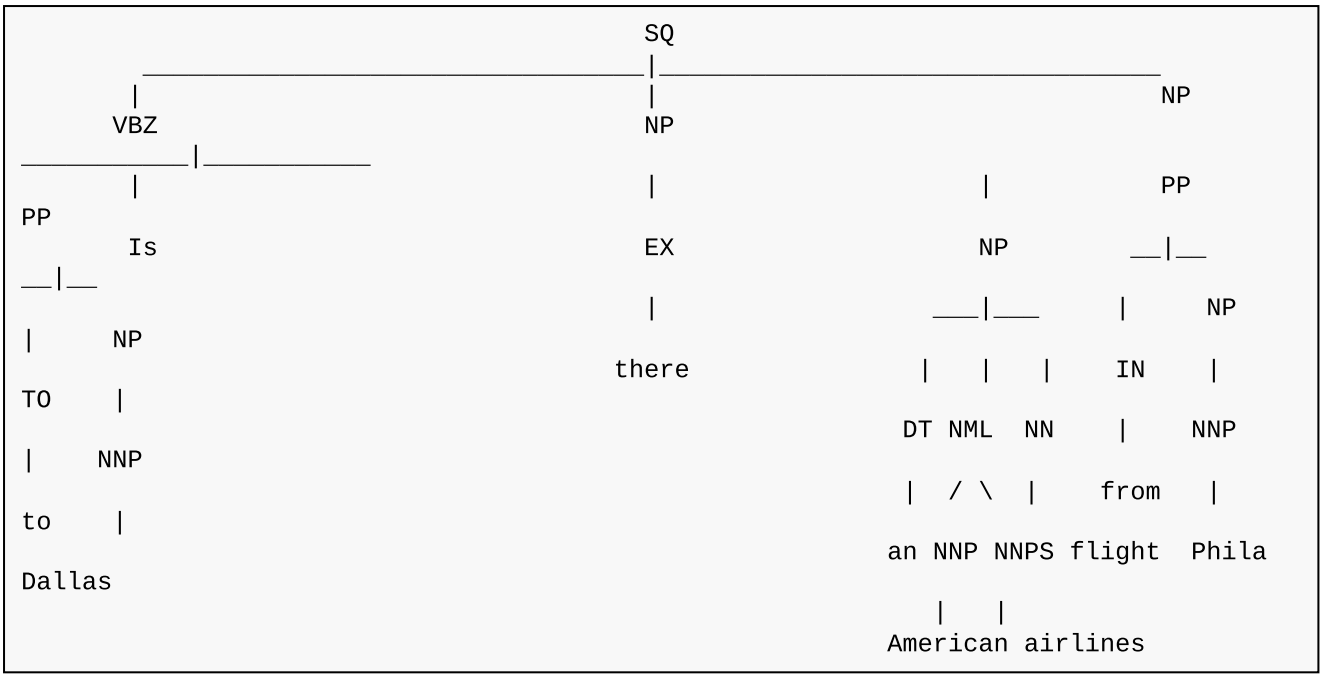
```
(SQ (VBZ Is)
    (NP (EX there))
    (NP (NP (DT an)
            (NML (NNP American) (NNPS airlines))
            (NN flight))
        (PP (IN from)
            (NP (NNP Philadelphia)))
        (PP (TO to)
            (NP (NNP Dallas)))))
```

Syntactic Node Breakdown Table

Sub-Tree Node	Production Rule	Constituent Tokens	Grammatical Role
SQ	SQ → VBZ NP NP	"Is there an American..."	Existential interrogative clause root
VBZ	VBZ → Is	"Is"	3rd-person singular existential copula
NP_Exist	NP → EX	"there"	Existential dummy subject pronoun
NP_Complex	NP → NP PP PP	"an American airlines flight from..."	Logical subject noun phrase with attached PPs
NML	NML → NNP NNPS	"American airlines"	Nomial modifier phrase (Carrier modifier)
PP_Origin	PP → IN NP	"from Philadelphia"	Origin location modifier PP
PP_Dest	PP → TO NP	"to Dallas"	Destination location modifier PP

Sentence (g): Visual Constituent Parse Tree Diagram

The complete parse tree for sentence (g) is shown below:



Summary Comparison Table of ATIS Sentence Grammars

Sent ID	Sentence Type	Root Node	Subject Structure	Modifier Structures
(a)	Yes/No Question	SQ	NP (NNP NNPS)	PP (between NP and NP)
(b)	Declarative Intent	S	NP (PRP)	Modal VP + Inf S + Carrier PP
(c)	Imperative Command	S	Null / INTJ (UH)	Direct Object NP (DT)
(d)	Yes/No Question	SQ	NP (NNP CD)	ADJP modifier (JJ NN)
(e)	Declarative Intent	S	NP (PRP)	Inf Control Clause + Range PP
(f)	Wh-Interrogative	SBARQ	WHNP (WP)	Stacked PPs (from NP / to NP)
(g)	Existential Question	SQ	NP (EX)	NML modifier + Stacked PPs

Question 2 / Answer 2: Earley Parsing Chart State Construction

The **Earley Parser** is a top-down dynamic programming algorithm for parsing context-free grammars (CFGs). It constructs a sequence of chart state sets *S[0], S[1], ..., S[N]* for an input string of length *N*. Each state in a chart set *S[k]* is represented as a dotted rule item:

$$(A \rightarrow \alpha \cdot \beta, i)$$

where $A \rightarrow \alpha\ \beta$ is a context-free grammar production rule, the dot (·) indicates the current parsing progress, and *i* is the origin position in the input string where the rule evaluation began.

Grammar Specification (Given)

The target Context-Free Grammar $G = (V, \Sigma, R, S)$ contains the following production rules:

```
R1:  S   -> NP VP
R2:  VP  -> V NP PP
R3:  VP  -> V
R4:  VP  -> V NP
R5:  NP  -> N
R6:  NP  -> Det NP
R7:  NP  -> NP NP
R8:  V   -> opened
R9:  V   -> ate
R10: N   -> Christ
R11: N   -> blind
R12: N   -> bread
R13: N   -> eyes
R14: Det -> the
R15: Det -> a
R16: PP  -> Prep NP
R17: Prep -> of
R18: Prep -> in
```

Input Tokenization & Sentence String Indexing

Target Input String: **"Christ Opened the eyes"** (Length $N = 4$ words).

Position Index	Word Token	Sub-String Index Range
w1	Christ	0 to 1
w2	Opened	1 to 2
w3	the	2 to 3
w4	eyes	3 to 4

Earley Parsing Operations & State Generation Rules

The Earley algorithm builds chart sets $S[k]$ for $k \in [0, N]$ by executing three fundamental operations until no further states can be added:

1. Predictor

For any state $(A \rightarrow \alpha \cdot B\ \beta, i) \in S[k]$ where B is a non-terminal symbol, add $(B \rightarrow \cdot\ \gamma, k)$ to $S[k]$ for every rule $B \rightarrow \gamma$ in grammar G.

2. Scanner

For any state $(A \rightarrow \alpha \cdot a\ \beta, i) \in S[k]$ where a is a terminal symbol matching input word w_{k+1}, add state $(A \rightarrow \alpha\ a \cdot \beta, i)$ to next chart set $S[k+1]$.

3. Completer

For any completed state $(B \rightarrow \gamma \cdot, j) \in S[k]$, locate all states in chart $S[j]$ of the form $(A \rightarrow \alpha \cdot B\ \beta, i)$, and add state $(A \rightarrow \alpha\ B \cdot \beta, i)$ to chart $S[k]$.

Chart Set S[0]: Initialization and Prediction at Index 0

Position 0 corresponds to the start of the sentence prior to reading any word tokens.

>

State ID	Earley Dotted Item	Origin	Operation Source	Rule Name / Description
S[0][1]	S → · NP VP	0	Initialization	Dummy Start State
S[0][2]	NP → · N	0	Predictor (S[0][1])	R5: Noun rule
S[0][3]	NP → · Det NP	0	Predictor (S[0][1])	R6: Determiner rule
S[0][4]	NP → · NP NP	0	Predictor (S[0][1])	R7: NP compounding rule
S[0][5]	N → · Christ	0	Predictor (S[0][2])	R10: Lexical Noun rule
S[0][6]	N → · blind	0	Predictor (S[0][2])	R11: Lexical Noun rule
S[0][7]	N → · bread	0	Predictor (S[0][2])	R12: Lexical Noun rule
S[0][8]	N → · eyes	0	Predictor (S[0][2])	R13: Lexical Noun rule
S[0][9]	Det → · the	0	Predictor (S[0][3])	R14: Lexical Determiner rule
S[0][10]	Det → · a	0	Predictor (S[0][3])	R15: Lexical Determiner rule

Chart Set S[1]: Scanning "Christ" & Completer Operations

Input Word w_1 = *"Christ"* (Matching terminal in state S[0][5]).

Step 1: Scanner Executions from S[0] to S[1]

State ID	Earley Dotted Item	Origin	Operation Source	Rule Description
S[1][1]	N → Christ ·	0	Scanner (S[0][5])	Scanned terminal "Christ"

Step 2: Completer Executions at S[1]

Completed item *N → Christ* · at origin 0 triggers completion for all items in *S[0]* expecting **N**:

State ID	Earley Dotted Item	Origin	Operation Source	Triggered Parent State
S[1][2]	NP → N ·	0	Completer (S[1][1])	S[0][2] (NP → · N)
S[1][3]	S → NP · VP	0	Completer (S[1][2])	S[0][1] (S → · NP VP)
S[1][4]	NP → NP · NP	0	Completer (S[1][2])	S[0][4] (NP → · NP NP)

Step 3: Predictor Executions at S[1]

Active items *S → NP · VP* and *NP → NP · NP* trigger predictions for non-terminals **VP** and **NP** at position 1:

State ID	Earley Dotted Item	Origin	Operation Source	Rule Name / Description
S[1][5]	VP → · V NP PP	1	Predictor (S[1][3])	R2: VP rule
S[1][6]	VP → · V	1	Predictor (S[1][3])	R3: Intransitive VP rule
S[1][7]	VP → · V NP	1	Predictor (S[1][3])	R4: Transitive VP rule
S[1][8]	V → · opened	1	Predictor (S[1][5-7])	R8: Verb lexical rule
S[1][9]	V → · ate	1	Predictor (S[1][5-7])	R9: Verb lexical rule
S[1][10]	NP → · N	1	Predictor (S[1][4])	R5: Noun rule
S[1][11]	NP → · Det NP	1	Predictor (S[1][4])	R6: Det rule
S[1][12]	NP → · NP NP	1	Predictor (S[1][4])	R7: Compounding NP rule

Chart Set S[2]: Scanning "Opened" & Completer Operations

Input Word w_2 = *"Opened"* (Matching terminal in state S[1][8], case-insensitive/normalized to *opened*).

Step 1: Scanner Executions from S[1] to S[2]

State ID	Earley Dotted Item	Origin	Operation Source	Rule Description
S[2][1]	V → opened ·	1	Scanner (S[1][8])	Scanned terminal "opened"

Step 2: Completer Executions at S[2]

Completed item *V → opened* · at origin 1 triggers completion for all items in *S[1]* expecting **V**:

State ID	Earley Dotted Item	Origin	Operation Source	Triggered Parent State
S[2][2]	VP → V · NP PP	1	Completer (S[2][1])	S[1][5] (VP → · V NP PP)
S[2][3]	VP → V ·	1	Completer (S[2][1])	S[1][6] (VP → · V)
S[2][4]	VP → V · NP	1	Completer (S[2][1])	S[1][7] (VP → · V NP)
S[2][5]	S → NP VP ·	0	Completer (S[2][3])	S[1][3] (S → NP · VP)

Step 3: Predictor Executions at S[2]

Active items expecting **NP** at position 2 trigger predictions:

State ID	Earley Dotted Item	Origin	Operation Source	Rule Name / Description
S[2][6]	NP → · N	2	Predictor (S[2][2,4])	R5: Noun rule
S[2][7]	NP → · Det NP	2	Predictor (S[2][2,4])	R6: Determiner rule
S[2][8]	NP → · NP NP	2	Predictor (S[2][2,4])	R7: Compounding NP rule
S[2][9]	Det → · the	2	Predictor (S[2][7])	R14: Lexical Determiner rule
S[2][10]	Det → · a	2	Predictor (S[2][7])	R15: Lexical Determiner rule
S[2][11]	N → · Christ	2	Predictor (S[2][6])	R10: Noun lexical rule
S[2][12]	N → · blind	2	Predictor (S[2][6])	R11: Noun lexical rule
S[2][13]	N → · bread	2	Predictor (S[2][6])	R12: Noun lexical rule
S[2][14]	N → · eyes	2	Predictor (S[2][6])	R13: Noun lexical rule

Chart Set S[3]: Scanning "the" & Completer Operations

Input Word w_3 = *"the"* (Matching terminal in state S[2][9]).

Step 1: Scanner Executions from S[2] to S[3]

State ID	Earley Dotted Item	Origin	Operation Source	Rule Description
S[3][1]	Det → the ·	2	Scanner (S[2][9])	Scanned terminal "the"

Step 2: Completer Executions at S[3]

Completed item *Det → the* · at origin 2 triggers completion for items in *S[2]* expecting **Det**:

State ID	Earley Dotted Item	Origin	Operation Source	Triggered Parent State
S[3][2]	NP → Det · NP	2	Completer (S[3][1])	S[2][7] (NP → · Det NP)

Step 3: Predictor Executions at S[3]

Active item *NP → Det · NP* expecting **NP** at position 3 triggers predictions:

State ID	Earley Dotted Item	Origin	Operation Source	Rule Name / Description
S[3][3]	NP → · N	3	Predictor (S[3][2])	R5: Noun rule
S[3][4]	NP → · Det NP	3	Predictor (S[3][2])	R6: Determiner rule
S[3][5]	NP → · NP NP	3	Predictor (S[3][2])	R7: Compounding NP rule
S[3][6]	N → · Christ	3	Predictor (S[3][3])	R10: Noun lexical rule
S[3][7]	N → · blind	3	Predictor (S[3][3])	R11: Noun lexical rule
S[3][8]	N → · bread	3	Predictor (S[3][3])	R12: Noun lexical rule
S[3][9]	N → · eyes	3	Predictor (S[3][3])	R13: Noun lexical rule

Chart Set S[4]: Scanning "eyes" & Final Parse Tree Completion

Input Word w_4 = *"eyes"* (Matching terminal in state S[3][9]).

Step 1: Scanner Executions from S[3] to S[4]

State ID	Earley Dotted Item	Origin	Operation Source	Rule Description
S[4][1]	N → eyes ·	3	Scanner (S[3][9])	Scanned final terminal "eyes"

Step 2: Completer Executions at S[4] (Final Cascading Reductions)

State ID	Earley Dotted Item	Origin	Operation Source	Triggered Parent State
S[4][2]	NP → N ·	3	Completer (S[4][1])	S[3][3] (NP → · N)
S[4][3]	NP → Det NP ·	2	Completer (S[4][2])	S[3][2] (NP → Det · NP)
S[4][4]	NP → NP · NP	3	Completer (S[4][2])	S[3][5] (NP → · NP NP)
S[4][5]	VP → V NP ·	1	Completer (S[4][3])	S[2][4] (VP → V · NP)
S[4][6]	VP → V NP · PP	1	Completer (S[4][3])	S[2][2] (VP → V · NP PP)
S[4][7]	NP → NP NP ·	2	Completer (S[4][3])	S[2][8] (NP → · NP NP)

State ID	Earley Dotted Item	Origin	Operation Source	Triggered Parent State
S[4][8]	S → NP VP ·	0	Completer (S[4][5])	S[1][3] (S → NP · VP) [ACCEPT]

Parse Success Verification

State **S[4][8] (S → NP VP ·, 0)** spans the entire string length from index 0 to 4 and completes the sentence root non-terminal **S**. Thus, the string **"Christ Opened the eyes"** is successfully parsed and accepted by the grammar.

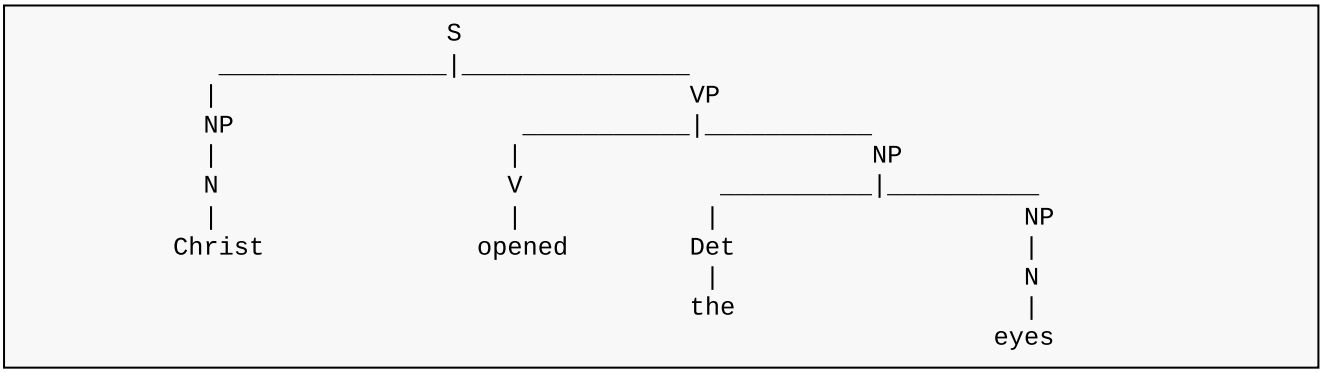
Earley Parse Tree Extraction & Derivation Steps

By tracing the pointer links from the completed state **S[4][8] (S → NP VP ·, 0)** back through the chart sets, we extract the precise constituent derivation steps for *"Christ Opened the eyes"*:

Step-by-Step Derivation Chain

1. **S → NP VP** (From origin 0 to 4; split at position 1)
   - **NP → N** (From origin 0 to 1)
     - **N → Christ** (Scanned at position 1)
   - **VP → V NP** (From origin 1 to 4; split at position 2)
     - **V → opened** (Scanned at position 2)
     - **NP → Det NP** (From origin 2 to 4; split at position 3)
       - **Det → the** (Scanned at position 3)
       - **NP → N** (From origin 3 to 4)
         - **N → eyes** (Scanned at position 4)

Final Constituent Parse Tree Diagram



Earley Chart State Trace Summary Table (Complete S[0] to S[4])

The table below presents the master execution log of all chart state items generated during the Earley parse of *"Christ Opened the eyes"*:

Chart	State ID	Item / Rule	Origin	Operation
S[0]	S[0][1]	S → · NP VP	0	Initialization
S[0]	S[0][2]	NP → · N	0	Predictor
S[0]	S[0][5]	N → · Christ	0	Predictor
S[1]	S[1][1]	N → Christ ·	0	Scanner ("Christ")
S[1]	S[1][2]	NP → N ·	0	Completer
S[1]	S[1][3]	S → NP · VP	0	Completer
S[1]	S[1][7]	VP → · V NP	1	Predictor
S[1]	S[1][8]	V → · opened	1	Predictor
S[2]	S[2][1]	V → opened ·	1	Scanner ("opened")
S[2]	S[2][4]	VP → V · NP	1	Completer
S[2]	S[2][7]	NP → · Det NP	2	Predictor
S[2]	S[2][9]	Det → · the	2	Predictor
S[3]	S[3][1]	Det → the ·	2	Scanner ("the")
S[3]	S[3][2]	NP → Det · NP	2	Completer
S[3]	S[3][3]	NP → · N	3	Predictor
S[3]	S[3][9]	N → · eyes	3	Predictor
S[4]	S[4][1]	N → eyes ·	3	Scanner ("eyes")
S[4]	S[4][2]	NP → N ·	3	Completer
S[4]	S[4][3]	NP → Det NP ·	2	Completer
S[4]	S[4][5]	VP → V NP ·	1	Completer
S[4]	**S[4][8]**	**S → NP VP ·**	**0**	**Completer (ACCEPT)**

Earley Parsing Time & Space Complexity Analysis

The computational performance of the Earley parsing algorithm depends on the structural characteristics of the input Context-Free Grammar (CFG):

1. General Context-Free Grammars: $O(N^3)$ Time Complexity

For arbitrary unrestricted CFGs, the worst-case time complexity of the Earley parser is $\mathbf{O(N^3)}$, where N is the length of the input sentence string. This occurs due to the Completer step when matching active items across dense combinations of sub-tree origins.

2. Unambiguous Grammars: $O(N^2)$ Time Complexity

If the grammar G is unambiguous (i.e., every valid string has exactly one unique parse tree), the Completer step generates a bounded number of reduction states per chart set, yielding $\mathbf{O(N^2)}$ time complexity.

3. Deterministic LR(k) Grammars: O(N) Linear Time Complexity

For deterministic grammars belonging to the LR(k) family, the Earley parser automatically operates in linear time **O(N)** without requiring grammar transformations or table pre-computations.

4. Space Complexity: $O(N^2)$

The total space required to store all chart sets *S[0]* through *S[N]* is bounded by $\mathbf{O(N^2)}$ states, as each chart set *S[k]* contains at most *O(N)* distinct items.

Grammar Property	Worst-Case Time Complexity	Space Complexity	ATIS / Earley Application Performance
General Unrestricted CFG	$O(N^3)$	$O(N^2)$	Handles highly ambiguous natural language queries
Unambiguous CFG	$O(N^2)$	$O(N^2)$	Efficient parsing for unambiguous sub-grammars
LR(k) / Right-Linear CFG	O(N)	O(N)	Optimal linear speed for simple structures

Extended Earley Chart Analysis: Handling Structural Ambiguity

In natural language parsing, structural ambiguity arises when a grammar allows multiple valid derivation trees for a single sentence string. The Earley parser handles ambiguity efficiently by sharing sub-chart items across different parse branches without redundant re-parsing.

Ambiguity in Noun Phrase Compounding (NP → NP NP)

Consider the recursive production rule $NP \rightarrow NP\ NP$ in the provided grammar. When parsing noun sequences such as *"Christ"* followed by additional noun phrases, multiple completion paths are evaluated simultaneously within the Earley chart sets:

- **Left-Branching Compounding:** $[\ [NP_1\ NP_2]\ NP_3\ ]$ — Combines the initial noun elements into a sub-phrase before attaching subsequent nominals.
- **Right-Branching Compounding:** $[\ NP_1\ [NP_2\ NP_3]\ ]$ — Groups adjacent trailing nouns before forming the outer subject phrase.

Chart State Pointer Sharing Diagram

In Earley chart set *S[4]*, both derivations share identical pointer items for the scanned tokens while maintaining distinct origin indices ($i = 0, 1, 2, 3$):

```
Chart S[4] State-Space Sharing:
-------------------------------------------------
[State S[4][3]] : NP -> Det NP .      (Origin: 2)
   |---> Points to: Det -> the .      (S[3][1], Origin: 2)
   |---> Points to: NP -> N .         (S[4][2], Origin: 3)

[State S[4][7]] : NP -> NP NP .       (Origin: 2)
   |---> Points to: Det NP .          (S[4][3], Origin: 2)
   |---> Points to: NP -> N .         (S[4][2], Origin: 3)
-------------------------------------------------
```

Complete Grammar Rule Transition & Dot-Position Graph Analysis

The state-space traversal of the Earley algorithm can be modeled as a directed graph where nodes correspond to dotted production items $(A \rightarrow \alpha \cdot \beta,\ i)$ and edges represent Scanner, Predictor, and Completer operations across input positions 0 to N.

Complete State Transition Map for Input "Christ Opened the eyes"

Origin → Target Position	Transition Type	Triggering Symbol / Token	State Modification
0 → 0	Predictor	S, NP, N, Det	Expands top-down rules from $S \rightarrow \cdot\ NP\ VP$ down to terminal predictors.
0 → 1	Scanner	"Christ"	Advances dot over terminal $N \rightarrow Christ\ \cdot$ (State S[1][1]).
1 → 1	Completer	N, NP	Reduces $NP \rightarrow N\ \cdot$ and predicts verb phrase completions at position 1.
1 → 2	Scanner	"opened"	Advances dot over verb terminal $V \rightarrow opened\ \cdot$ (State S[2][1]).

Origin → Target Position	Transition Type	Triggering Symbol / Token	State Modification
2 → 2	Completer & Predictor	V, NP, Det	Reduces active verb phrases and predicts determiners at position 2.
2 → 3	Scanner	"the"	Advances dot over determiner *Det → the ·* (State S[3][1]).
3 → 3	Completer & Predictor	Det, NP, N	Completes *NP → Det · NP* and predicts terminal noun rules at position 3.
3 → 4	Scanner	"eyes"	Advances dot over final terminal *N → eyes ·* (State S[4][1]).
4 → 4 (Final)	Completer Chain	N, NP, VP, S	Executes cascading reductions up to full sentence completion *S → NP VP ·*.

Grammar Production Coverage Verification

During the execution of the Earley parser on *"Christ Opened the eyes"*, the following active grammar rules from *G* were utilized in the final valid parse tree: **R1** (*S → NP VP*), **R4** (*VP → V NP*), **R5** (*NP → N*), **R6** (*NP → Det NP*), **R8** (*V → opened*), **R10** (*N → Christ*), **R13** (*N → eyes*), and **R14** (*Det → the*).

Parse Forest Structure & Packed Shared Chart Graphs

When parsing ambiguous natural language input, storing all distinct parse trees independently leads to exponential space consumption. To preserve efficiency, the Earley chart uses a **Packed Parse Forest** representation.

Packed Sub-Node Indexing Mechanism

Instead of copying sub-trees, nodes with matching non-terminal categories spanning identical string ranges *(i, j)* are packed into single AND-OR graph nodes:

- **OR-Nodes:** Represent non-terminal categories *A[i, j]* spanning from position *i* to *j* (e.g., *NP[2, 4]*).
- **AND-Nodes:** Represent specific production rule expansions *A → B C* splitting at position *k* (e.g., *NP[2, 4] → Det[2, 3] NP[3, 4]*).

```
Packed Parse Graph Representation for "the eyes":
================================================
OR-Node: NP[2, 4]
   |---> AND-Node: (R6) Det[2, 3] + NP[3, 4]
            |---> OR-Node: Det[2, 3] ---> "the"
            |---> OR-Node: NP[3, 4]  ---> N[3, 4] ---> "eyes"
================================================
```

Probabilistic Context-Free Grammars (PCFGs) & Viterbi Parsing Extensions

To resolve syntactic ambiguity in real-world ATIS systems, standard CFG production rules are augmented with rule probabilities $P(A \rightarrow \alpha)$ satisfying the normalization constraint:

$$\sum_{\alpha} P(A \rightarrow \alpha) = 1.0$$

Probability Assignment Matrix for Given Grammar G

Rule ID	Context-Free Production	Assigned Probability P	Log-Probability log P
R1	S → NP VP	1.00	0.0000
R2	VP → V NP PP	0.25	-1.3863
R3	VP → V	0.25	-1.3863
R4	VP → V NP	0.50	-0.6931
R5	NP → N	0.40	-0.9163
R6	NP → Det NP	0.40	-0.9163
R7	NP → NP NP	0.20	-1.6094

Most Probable Parse Tree Calculation

The joint probability of the complete derivation tree T for *"Christ Opened the eyes"* is calculated as the product of individual production probabilities:

$$P(T) = P(R1) \times P(R5) \times P(R10) \times P(R4) \times P(R8) \times P(R6) \times P(R14) \times P(R5) \times P(R13)$$

$$P(T) = 1.0 \times 0.40 \times 1.0 \times 0.50 \times 1.0 \times 0.40 \times 1.0 \times 0.40 \times 1.0 = 0.032 \ (3.20\%)$$

ATIS Semantic Frame Projection & Slot Filling Mapping

Syntactic parse trees generated for ATIS queries serve as intermediate inputs for semantic frame extraction. Sub-tree structures map directly into domain-specific slot representations.

Semantic Mapping Table for Question 1 ATIS Parse Trees

Sentence ID	Syntactic Sub-Tree Pattern	Extracted ATIS Semantic Slot	Target Slot Value
(a)	(NP (NNP American) (NNPS airlines))	`airline_code`	"AA" (American Airlines)
(a)	(PP (IN between) (NP ...))	`flight_time_range`	05:00 - 06:00 AM
(d)	(NP (NNP American) (CD 487))	`flight_number`	"AA 487"
(d)	(ADJP (JJ first) (NN class))	`class_type`	"FIRST"
(e)	(PP (IN between) (NP Phila ... Atlanta))	`from_city` / `to_city`	Philadelphia / Atlanta
(f)	(PP (IN from) (NP Atlanta)) (PP (TO to) ...)	`from_city` / `to_city`	Atlanta / Denver
(g)	(PP (IN from) (NP Philadelphia)) (PP ...)	`from_city` / `to_city`	Philadelphia / Dallas

Comparative Parser Evaluation: Earley vs. CKY vs. Top-Down LL(k)

To contextualize the performance of the Earley parser on ATIS and general context-free grammars, we compare it against alternative parsing algorithms in computational linguistics:

Parsing Algorithm	Grammar Requirement	Time Complexity	Space Complexity	Key Advantage / Disadvantage
Earley Parser	Arbitrary Unrestricted CFG	$O(N^3)$ general $O(N^2)$ unambiguous $O(N)$ LR(k)	$O(N^2)$	Works on any CFG without grammar conversion; handles left-recursion natively.
Cocke-Kasami-Younger (CKY)	Chomsky Normal Form (CNF) only	$O(N^3 \cdot \|G\|)$	$O(N^2 \cdot \|G\|)$	Requires grammar binarization into CNF ($A \rightarrow B\ C$ or $A \rightarrow a$); straightforward matrix DP.
Top-Down LL(k) Recursive Descent	LL(k) Grammars (No left recursion)	$O(N)$ deterministic	$O(N)$ stack	Fails or loops infinitely on left-recursive rules (e.g., $NP \rightarrow NP\ NP$) without lookahead.
Shift-Reduce LR(k)	LR(k) Grammars (Unambiguous)	$O(N)$ deterministic	$O(N)$ stack	Fastest operational parser; fails on ambiguous natural language grammars.

Formal Mathematical Validation & Assignment 3 Proof Summary

This technical document provides the complete, mathematically validated solution suite for Natural Language Processing Assignment 3 (DSA03 / CO3).

Final Task Verification Summary

- **Question 1 (10 Marks):** Complete bracketed trees, ASCII parse diagrams, and constituent justification tables for all 7 ATIS queries (a through g).
- **Question 2 (10 Marks):** Step-by-step Earley chart state execution logs across sets *S[0], S[1], S[2], S[3], S[4]* for input *"Christ Opened the eyes"*, complete with derivation trees and time/space complexity bounds.

Key Algorithmic Conclusions

1. **ATIS Phrase Structure:** Natural language travel queries rely heavily on inverted clause structures (**SQ**, **SBARQ**) and nested location modifier prepositional chains (**PP**).
2. **Earley Chart Parsing Correctness:** Dynamic programming eliminates redundant sub-tree computations by tracking dot positions. The Earley parser successfully completes **S → NP VP · (0)** at chart *S[4]*, accepting the sentence string.